Midterm #1 - Spring 2024

1. We start with $S_0 = \{q_0\}$

as there are no ε transitions would require more moves to get out

$S_0 \xrightarrow{a,b} S_1$, we go to q_1 or q_2

$S_1 = \{q_1, q_2\}$

We see that from q_1 we can get to q_3 (the accepting state) by taking

a, b or c or c, b

S_0 S_3 can be arrived at from S_1 taking ε, b while S_5 can be arrived at from S_1 with c and S_6 can be arrived at from c, b

Finally with $q_0 \xrightarrow{b} q_1 \xrightarrow{\varepsilon} q_2 \xrightarrow{b} q_3 \xrightarrow{b} q_3$ we can arrive at the only other S_7 in similar light, $s_0 \xrightarrow{b} S_1 \xrightarrow{\varepsilon} S_3 \xrightarrow{b} S_7$

— Are only accepting states.

2.a) E = lowest precedence (^)
T = middle precedence (&)
F = highest precedence (~)

So...

E -> E ^ T | T
T -> T & F | F
F -> ~F | id

b) we can write

A_SJC -> SJC -> A_LAX
A_SJC -> SJC -> A_DFW
A_LAX -> LAX -> A_SJC
A_LAX -> LAX -> A_DFW
A_LAX -> LAX -> A_ORD
A_DFW -> DFW -> A_SJC
A_DFW -> DFW -> A_LAX
A_DFW -> DFW -> A_ORD
A_DFW -> DFW -> A_SFO
A_ORD -> ORD -> A_LAX
A_ORD -> ORD -> A_DFW
A_ORD -> ORD -> A_SFO
A_SFO -> SFO -> A_DFW
A_SFO -> SFO -> A_ORD
A_SFO -> SFO

The A prefix explains which airport we are currently at

3a) First (S) = {a}

First (E) = {c, ε}

First (Q) = {a, ε}

b) Follow (S) = {$}

Follow (E) = {$} ← These are subsets of S

Follow (Q) = {$}

c) No b/c S->aE and S->abQ both start with a terminal a, so a one token lookahead can't decide which production to use

d) Second (S) = {c, b, ε} find first of a:

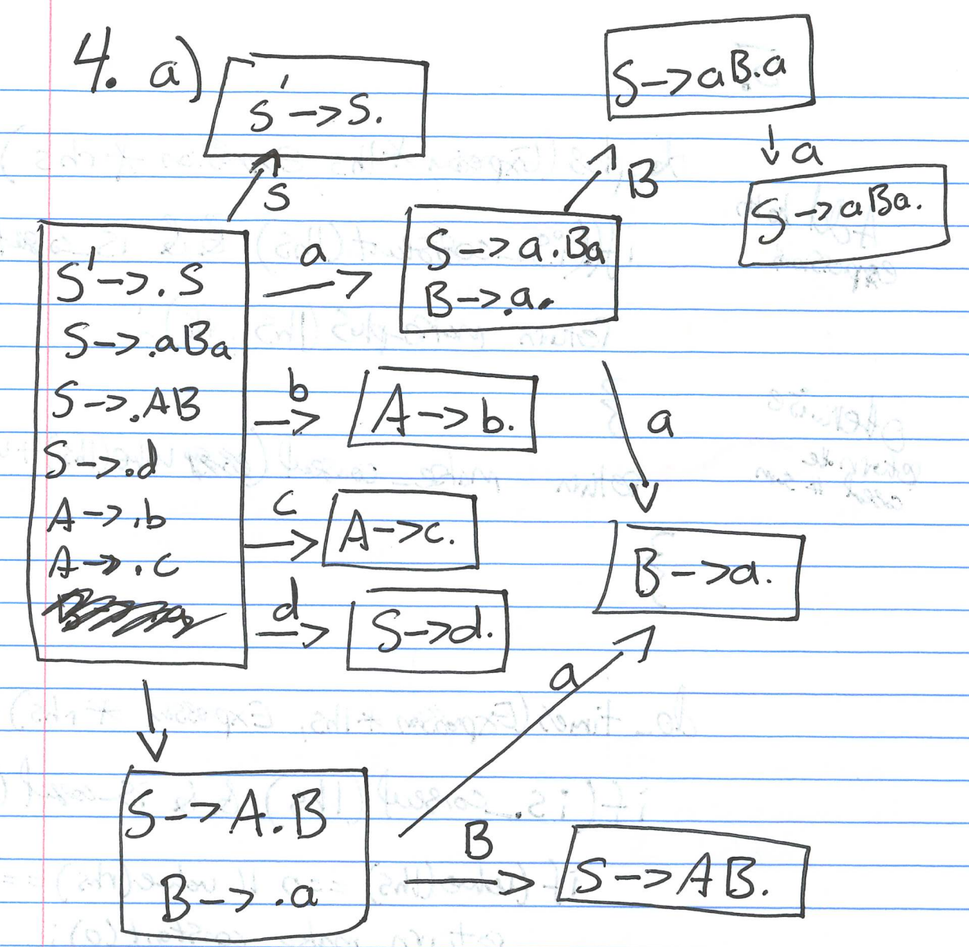
Second (E) = {a, b, ε} First(a) = {c, b, ε}

Second (Q) = {c, [illegible] ε}

e)

t_1	a				b				c				$
t_2	a	b	c	$	a	b	c	$	a	b	c	$	
S	abQ		dE	dE									
E									cQ	cb	cQ		ε
Q			dE	dE									ε

4. a)



b) If any no-lookahead state has conflict, not LR(0). We see that there are no reduce-reduce or shift-reduce conflicts, resolved

c) Yes valid same reasoning as above (b).

5.

Add both expressions

Otherwise returning the const # sum

```
do_plus(Expression *lhs, Expression *rhs){
  if((is_constant(lhs) && is_const(rhs))){
    return make_plus(lhs, rhs);
  }
  return make_const(value(lhs)+value(rhs));
}
```

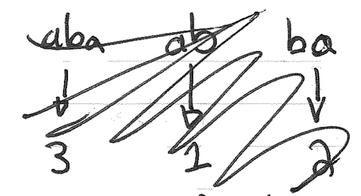
```
do_times(Expression *lhs, Expression *rhs){
  if(is_const(lhs) && is_const(rhs)){
    if(value(lhs) == 0 || value(rhs) == 0){
      return make_constant(0);
    }
    return make_const(value(lhs)*value(rhs));
  }
  return make_times(lhs, rhs);
}
```

Midterm 2 - Spring 2025

1) a) B/c it must accept the start state (ε) and then the state after having read "a" which means it must have at least two states

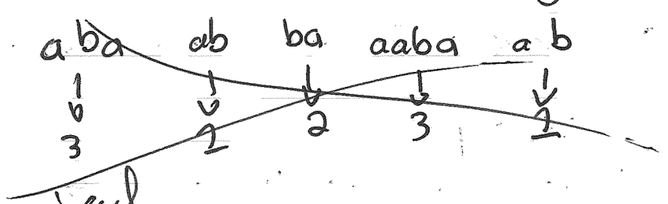
b) You make some new accepting state and then add ε-transition to said new accepting state

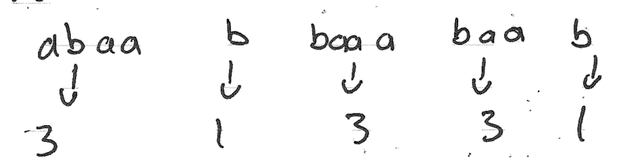
2.



b/c * means zero or more occurs with ε included

we also want the one ending in aa





3)

EPSILON

$$nullable = true
$$ beginnings = {}

CHAR

$$nullable = false
$$ beginnings = {$1.val}

~~expr '(' expr ')'~~

expr expr
if($1nullable)
$$ beginnings = $1beginnings
else
$$ beginnings = $1beginnings ∪ $2beginnings
$$nullable = $1nullable && $2nullable

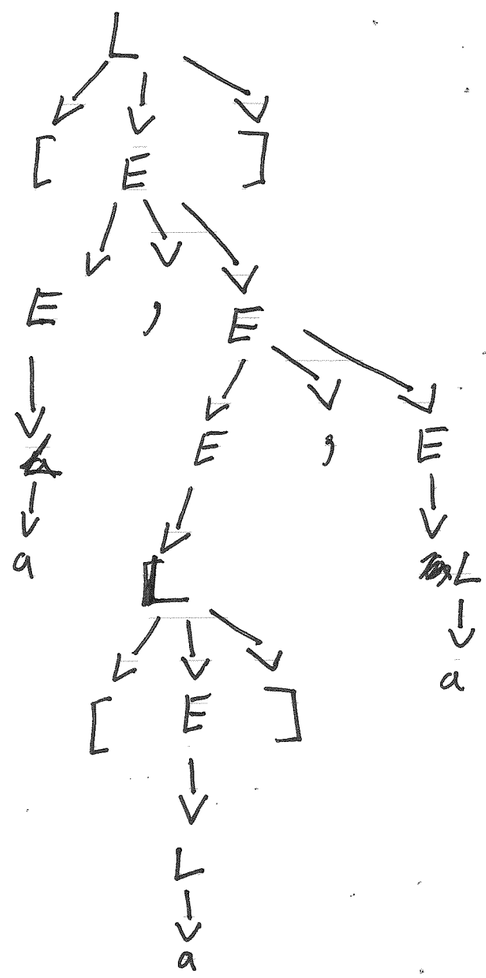
expr '+' expr
$$ nullable = $1nullable || $3nullable
$$ beginning = $1beginnings ∪ $3 beginnings

expr '*'
$$nullable = true
$$beginnings = $1 beginnings

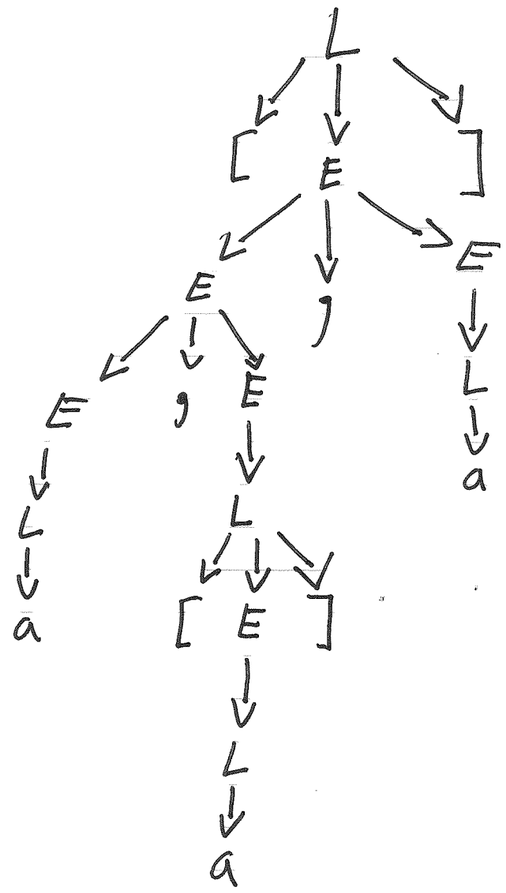
'(' expr ')'

$$nullable = $2nullable
$$ beginnings = $2 beginnings

4) a)



We can draw a second parse tree going to "the left" instead of to "the right"



b) First (L) = {[, a}, First (E) = {ε, ,, [, a}
Follow (E) = {,,], $}
Follow (L) = {,,], $}

c) We have First(E,E) = First (E) = {[, a, ε}
and First (L) = {[, a} so LL(1) which sees "[" or "a" doesn't know which one to choose.

d) L -> [T] | a
T -> T, E | E
E -> L | ε

L -> [E] | a
E -> L T | T
T -> , E T | ε

5) Really off State 0, we can see that
A -> xA
A -> y

Midterm 2026

1. ababa ab bb aba

↓ ↓ ↓ ↓

4 1 3 2

b/c there are less substrings required, just one

2. a) Yes it is b/c P and Q are interchangable. we can replace both P and Q back to back and get same e.g.

P a P b and P a P b

↓ ↓ ↓ ↓ ↓ ↓

a a Q a

↓

P

↓

a

which are two dif trees but output is the same.

b) four letters with _a_b and _b_a etc.

c) L(G) is regular b/c it is a finite language as it generates the four character strings. Thus it's regular.

as seen above

3) Node* handle_not(Node* e){

if(e.getType() == And){

return make_or(make_not(e.getLhs()), make_not(e.getRhs()));

}else if(e.getType() == Or){

return make_and(make_not(e.getLhs()), make_not(e.getRHS()));

}else if(e.getType() == Not){

return e.getOpnd();

}else if(e.getType() == Var){

return make_not(e);

}

}

4) a) Follow(S) = {$} b/c S is Start and nothing else in the grammar can appear after S

we can also see that A, b = ε and we know for LL(1) that if A -> ε put all ε-rules into the table. So...

Follow(A) = {b}

4. b) Ready from the table for A we see:

aA, ε, c

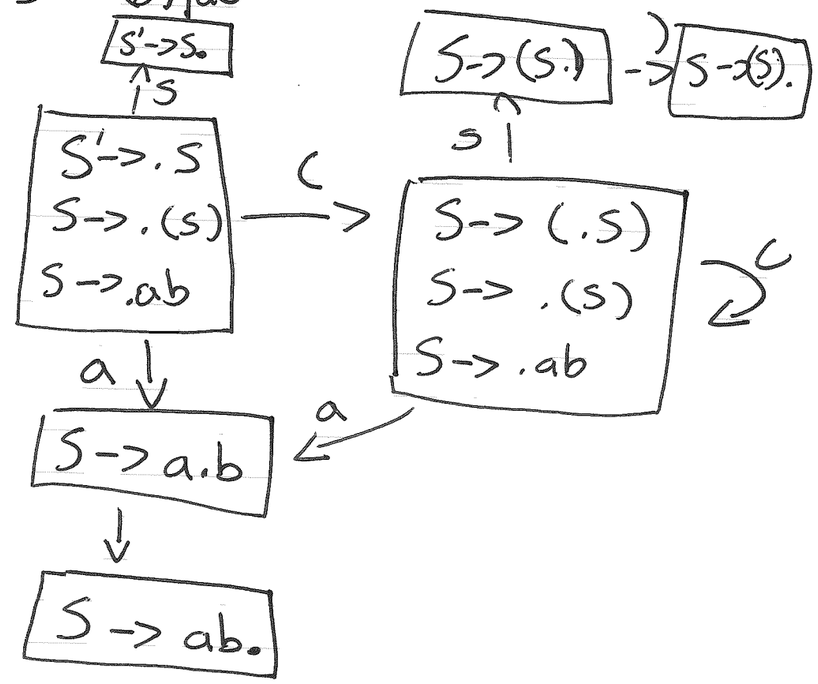
c) we see that $[S, c] = Ab$ b/c $c \in FIRST(Ab)$, so Ab goes in c column.

d) we see that $[S, b] = Ab$

5a) Taken as a given as no DFA parsing table is given in this midterm-2026

$S \rightarrow (S) \mid ab$

b)



Final 2024

1a) We know that $e_0.f(e_1,\dots,e_n)$ tells us that we call f on e_0 with $e_1,\dots,e_n$ as args

The key is that we ~~evaluate~~ Eck e_0 first

$$S_0,\ S_1,\ E \vdash e_0 \to V_0,\ S_2$$
$$V_0' = X(a_1 = la_1, \dots, a_m = la_m$$

⋮ Rest of the operand semantics

$$S_0, S_1, E \vdash e_0?.f(e_1,\dots,e_n) \mapsto V_{n+1}, S_{n+1}$$

b) Simpler b/c the entire expression returns void when the target object becomes void

$$S_0,\ S_1,\ E \vdash e_0 \to \text{void},\ S_2$$

$$S_0,\ S_1,\ E \vdash e_0?.f(e_1,\dots,e_n) \to \text{void}, S_2$$

#2. x is originally at 8($sp) then he adding 8 moves it down by 8, only 16($sp) for the first

In a similar logic, we can see that

4 ($sp)
12 ($sp)
4 ($sp)

Then filly at for label 0, 2, at 2 we get

4 ($sp)
8 ($sp)
4 ($sp)
4 ($sp)

#3. a)

Array

$C \vdash e_1 : T_1$
$C \vdash e_2 : T_2$
$\vdots$
$C \vdash e_n : T_n$
$T = lub(T_1, T_2, \ldots, T_n)$

$C \vdash [e_1, e_2, \ldots, e_n] : [T]$

Least upper bound, the closest shared parent

Array-Index

$C \vdash e_1 : [T_1]$
$C \vdash e_2 : Int$

$C \vdash e_1[e_2] : T_1$

Array then first indexed

[Array-Concat]

$C \vdash e_1 : [T_1]$
$C \vdash e_2 : [T_2]$
$T = lub(T_1, T_2)$

$C \vdash e_1 + e_2 : [T]$

b) No, even if chess ≤ boardGame, allow [chess] ≤ [BoardGame] would let someone treat [chess] array as a [BoardGame] array & insert a non-chess BoardGame obj. Then, once the original [chess] array gets read the cmpler will expect every element to be chess, but one element could just be BoardGame which breaks type safety

#4.

a)

1. a=1
2. b=1
3. c=1
4. e=a+b
5. d=1
6. d=c+e
7. d=d+e

Boards create the variable interferences

(working backwards from e being live at exit)

b) b/c of a, b, c being connected, we need to have three colors there, so K=3

Fewest neighbors + first in alphabetical order: d

c
b
a
e
d

first

c) r_1 : a=1
r_2 : b=1
r_3 : c=1
r_1 : e= a+b

r_2 is free

r_2 : d = 1

r_2 : d=c+e

r_2 : d=d+e

All registers now (at end) can be freed except r_2

5. BB1: 1-3
a) BB2: 6-14
BB3: 15-16
BB4: 4-5

b) (BB1, BB4)
(BB4, BB2)
(BB4, BB3)
(BB2, BB4)

c) We can see that c=0 then c=c+1 which violates the SSA rule/property as we are assigning the variable twice

d) We can elimate f=e line 7 as it gets used again in line 11 and just use e instead of defining a new variable e

we can also move 13 higher up to replace line 8. B/c no other BB all use i but is c is already incrubed we can replace it with e

Also g, h, and j serve no purpose in any future block here, so those can be eliminated.

Final 2025

1.a) Student is a subtype of Person. I.e. anywhere Student gets used so can Person.

① Student->Student ≤ Student->Person

Super if it can accept the broader input type Person and return the narrow type Student

② Person -> Student ≤ Student -> Student
combing ① and ②

③ Person -> Student ≤ Student -> Person

④ Person -> Person ≤ Student -> Person
combing ③ and ④

⑤ Person -> Student ≤ Person -> Person

b) For A must be broader than A' and B must be narrower than B' b/c of

$A' \leq A$

$B \leq B'$

Question 2

a)

```
if (t1 ≠ 0) goto (start);
```

start: expression 2

Execute expression 2 if $t1 \neq 0$

b)

$$\frac{C \vdash e_1 : T \qquad C \vdash e_2 : Int}{C \vdash \text{do } e_1 \text{ while } e_2 : T}$$

checks it now as a do-while loop

c)

$$\frac{C \vdash e_1 : T \qquad C \vdash e_2 = 0}{C \vdash e_1 \text{ while } e_2 : T}$$

When the integer is zero, we go to the false case

Giving

$$\frac{C \vdash e_1 : T \qquad C \vdash e_2 : T \qquad C \vdash \text{do } e_1 \text{ while } e_2 : T}{C \vdash \text{do } e_1 \text{ while } e_2 : T}$$

when we execute the true case, the integer is not zero

#3.a) B/c if it's not in SSA form, then we can't replace replace those variables/values with the previously defined values as the assignment might change said value.

```
a = 0
b = a+1
a = 1
c = a+1
```

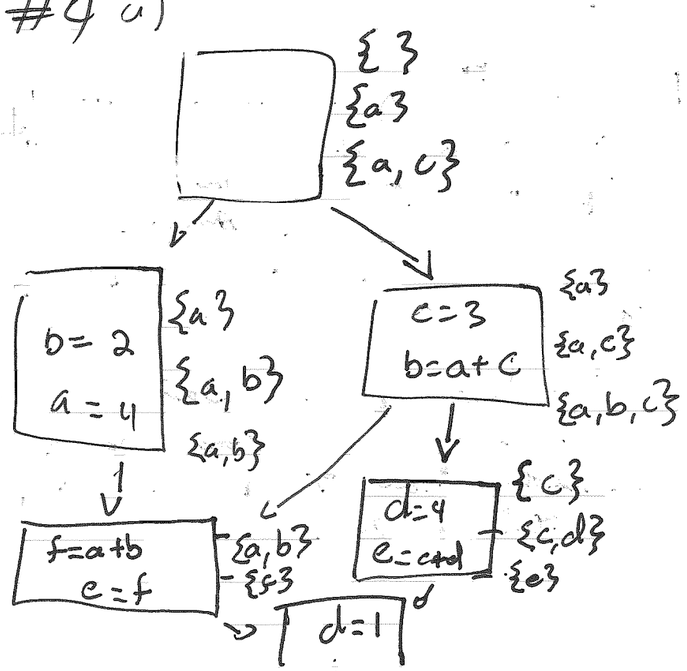
So we can't replace a+1 with b as the value of a changes

b)

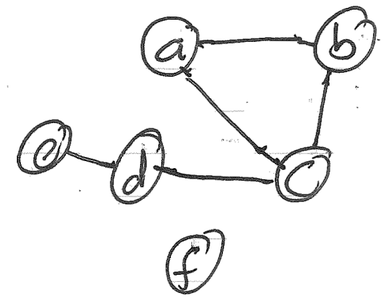
```
x = y + 0
z = x
return z
```

we can see that folding it first turns the expression into a simple copy then propagating removes it completely after.

#4 a)



b)



c)

r_1 : a, f, e

r_2 : c

r_3 : b, d

#5 a) When there are a few objects alive, stop+copy touches and copies only live obj while mark-and-sweep scans entire heap to find all free unreachable objects

b) When heap memory is almost full. B/c need to allocate new memory (e.g. disk) To temporary copy reachable mem before free everything else seems very costly rather than doing a single scan after the marks

c) If low pause time matters, e.g. mobile apps where can only frees objects immediately instead of stopping the whole program for a full heap scan.

#6.

We can add some inheritance relations to get

```
class A {
  x: Int <- 1
  h() : Int {
    0
  }
}
```

```
Class B inherits A {
  y: Int <- 2;
}
```

```
class C inherits B {
  f() : Int {
    0
  }
}
```

Final 2026

a) $O, M, C \vdash e_1 : Int$
$O, M, C \vdash e_2 : Int$
$O, M, C \vdash e_3 : T$

Grammar Production

$O, M, C \vdash$ for ID <- e_1 to e_2 loop e_3 pool : Object

b) Initializing the for-loop

$S_0, S_1, E \vdash e_1 \rightarrow Int(i), S_2$
$l = newloc(S_2)$
$S_3 = S_2[Int(i)/l]$
$S_0, S_3, E \vdash forloop(ID, e_2, e_3) \rightarrow void, S_4$

$S_0, S_1, E \vdash$ for ID <- e_1 to e_2 loop e_3 pool $\rightarrow$ void, S_4

Continue-loop

$S_1(E(ID)) = Int(i)$
$S_0, S_1, E \vdash e_2 \rightarrow Int(j), S_2$
$i \leq j$
$S_0, S_2, E \vdash e_3 \rightarrow v, S_3$
$S_4 = S_3[Int(i+1)/E(ID)]$
$S_0, S_4, E \vdash forloop(ID, e_2, e_3) \rightarrow Void, S_5$

$S_0, S_1, E \vdash forloop(ID, e_2, e_3) \rightarrow void, S_5$

Stop-loop

$S_1(E(ID)) = Int(i)$
$S_0, S_1, E \vdash e_2 \rightarrow Int(j), S_2$
$i > j$

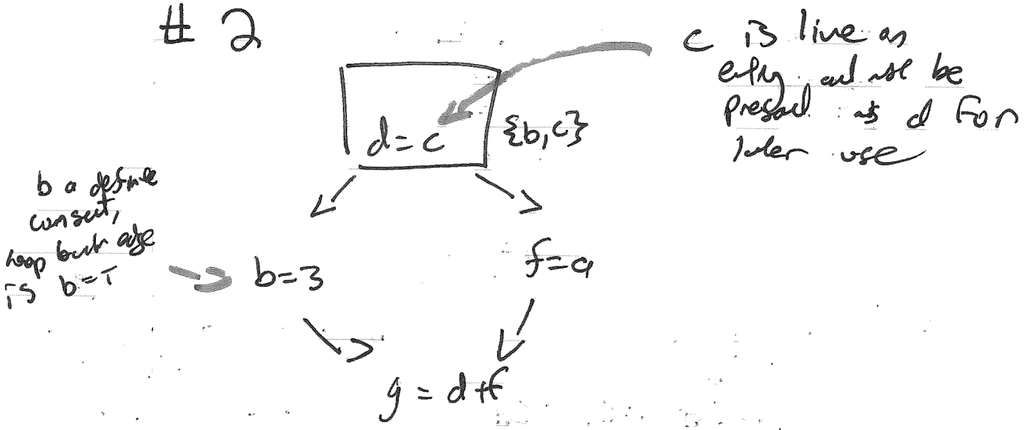
$S_0, S_2, E \vdash forloop(ID, e_2, e_3) \rightarrow Void, S_2$

c) cgen(for ID <- e_1 to e_2 loop e_3 pool) =
cgen(e_1)
push($a0)
bind ID -> 4($sp)
L:
lw $t1 4($sp)
lw $t2 12($a0)

cgen(e_2)
lw($t2 12($a0)
bgt $t1 $t2 Done
cgen(e_3)
lw $a0 4($sp)
add_int_const($a0, 1)
sw $a0 4($sp)

Done:
pop() Pop/remove temporary

2



#3

```
a = b + c
~~d = a + b~~
d = a + b
```

we can see here that before a=b+c, variable b and c are live, making the b — c edge before the second line where the a—b edge with d then getting isolated as it's only live after the last assignment.

#4

a)
```
a=new
b=new
c=new
d=new
e=new
a.a=e
a.b=b
a.d=d
b=null
c=null
d=null
e=null
```

b)
```
a=new
b=new
c=new
d=new
c=null
```

c)
```
a=new
b=new
c=new
b.a=c
c.a=b
b=null
c=null
```

#5
```
class Main {
  x: Int;
  f(y:Int, z:Int):Int {
    if 0<y then
      x<- x+z
    else
      0
    fi
  };
};
```

B/c the assembly first 0<y then is the true branch copies and stores x+z back into the object x while the false branch leaves 0